Informe TP_U4 - FNBC y Desnormalización Controlada (Food Store)

Parte 1 - Análisis y aplicación de FNBC: ControlLoteAlmacen

(a) Dependencias funcionales (notación formal)
Sean:
- LoteID = lote_id
- DepositoID = deposito_id
- ResponsableControlID = responsable_control_id

A partir de la regla de negocio:
- Para un lote y un depósito interviniente dados, el responsable de control queda unívocamente determinado.
- Cada responsable de control pertenece, como dato maestro de la dotación de personal, a un único depósito.

Se desprenden:
$$
DF_1: \{LoteID, DepositoID\} \to \{ResponsableControlID\}
$$
$$
DF_2: \{ResponsableControlID\} \to \{DepositoID\}
$$

(b) Clausura, claves candidatas, atributos primos/no primos
Conjunto de atributos $R = \{LoteID, DepositoID, ResponsableControlID\}$.

Clausuras relevantes:
- $(\{LoteID, DepositoID\})^+ = \{LoteID, DepositoID, ResponsableControlID\}$
- $(\{LoteID, ResponsableControlID\})^+ = \{LoteID, ResponsableControlID, DepositoID\}$ (por DF_2)
- $(\{ResponsableControlID\})^+ = \{ResponsableControlID, DepositoID\}$

Clave candidata: $\{LoteID, DepositoID\}$ (también se puede derivar $\{LoteID, ResponsableControlID\}$ como clave candidata, dado que determina a todos los atributos).

Atributos primos: $LoteID, DepositoID, ResponsableControlID$ son primos en el sentido de pertenecer a alguna clave candidata.
Atributos no primos: Ninguno en este esquema (todos son primos). Sin embargo la violación es por DF_2 con determinante no superclave.

(c) ¿Cumple FNBC? Justificación
La Forma Normal de Boyce-Codd (FNBC) exige que para toda dependencia funcional no trivial $X \to Y$, X sea una superclave.

En `control_lote_almacen` con clave candidata $K=\{LoteID, DepositoID\}$:
- $DF_1: K \to \{ResponsableControlID\}$: el determinante K es superclave → cumple para DF_1.
- $DF_2: \{ResponsableControlID\} \to \{DepositoID\}$: el determinante $\{ResponsableControlID\}$ **NO** es superclave (su clausura no contiene todos los atributos de la relación), y la dependencia es no trivial.

Conclusión: `control_lote_almacen` **NO** está en FNBC.

(d) Anomalías (con la instancia provista)
Instancia:
```
(501, 30, 801)
(502, 30, 801)
(503, 31, 802)
```

- **Anomalía de inserción:** No se puede registrar que el responsable `801` está asignado al depósito `30` sin asociarlo a algún `lote_id` existente (o se fuerza una tupla artificial). Además, no se puede representar la asignación `(responsable, depósito)` de forma independiente.
- **Anomalía de borrado:** Si se elimina la única tupla que relaciona `lote_id=501` con `(deposito_id=30, responsable_control_id=801)`, se puede perder inadvertidamente la información de que `801` pertenece al

depósito `30` (dependencia de negocio del dato maestro).
- **Anomalía de actualización:** Si `801` cambiara de depósito (violando la regla "pertenece a un único depósito" o para corregir un error), habría que actualizar todas las tuplas donde aparece `801` `(501,30,801)` y `(502,30,801)`, con riesgo de inconsistencia si no se actualizan todas.

(e)-(f) Descomposición sin pérdida, vista de compatibilidad y migración
Se aplica descomposición en dos esquemas:

R1 (responsable_deposito):
```sql
CREATE TABLE responsable_deposito (
   responsable_control_id  BIGINT NOT NULL,
   deposito_id           BIGINT NOT NULL,
   PRIMARY KEY (responsable_control_id)
);
```
R2 (control_lote):
```sql
CREATE TABLE control_lote (
   lote_id               BIGINT NOT NULL,
   responsable_control_id BIGINT NOT NULL,
   PRIMARY KEY (lote_id, responsable_control_id),
   CONSTRAINT fk_control_lote_resp FOREIGN KEY (responsable_control_id)
      REFERENCES responsable_deposito(responsable_control_id)
);
```

Vista de compatibilidad:
```sql
CREATE OR REPLACE VIEW v_control_lote_almacen AS
SELECT cl.lote_id, rd.deposito_id, cl.responsable_control_id
FROM control_lote cl
JOIN responsable_deposito rd
  ON cl.responsable_control_id = rd.responsable_control_id;
```

Justificación unión sin pérdida: El atributo común es `responsable_control_id`. En `responsable_deposito` es **clave primaria** (superclave), en `control_lote` es **clave foránea** hacia ella. Dado que $DF_2: \{responsable_control_id\}\to\{deposito_id\}$ y `responsable_control_id` es clave en R1, el JOIN por clave garantiza que cada tupla de R2 se une con **exactamente una** tupla de R1, reconstruyendo sin duplicados ni pérdida la terna original $(lote_id, deposito_id, responsable_control_id)$.

Migración verificada: Los datos se migraron desde `control_lote_almacen` hacia las tablas descompuestas y la equivalencia se verificó mediante `v_control_lote_almacen` (coincide con la instancia original).

Parte 2 - Consulta con desnormalización controlada

(a) EXPLAIN ANALYZE antes
Consulta original (filtra por `ped.fecha = CURRENT_DATE`, `dp.eliminado = FALSE`, `ped.eliminado = FALSE`, agrupa por `c.nombre`, ordena por `SUM(dp.subtotal)` desc, `LIMIT 5`).

Observación: Con un volumen razonable, el plan tiende a dominar por **uniones** (`JOIN` entre `detalle_pedido`, `producto`, `categoria`, `pedido`), acceso por filtros sobre `pedido.fecha`/flags de eliminado y **agregación/ordenamiento** para obtener el Top 5 por monto vendido. El nodo de mayor costo es típicamente el **Hash Join** o **Nested Loop** según índices/cantidad, con trabajo de agrupación (`GroupAggregate`/`HashAggregate`) y `Sort` para el ordenamiento.

(b) Patrón elegido y justificación
Patrón: **Columna precalculada con disparador (trigger-based materialization)** — tabla `ventas_categoria_dia(fecha, categoria_id, categoria_nombre, total_vendido, cantidad_items, cantidad_pedidos, actualizado_en)` con clave primaria `(fecha, categoria_id)`.

Justificación:
- **Evidencia que motiva:** La consulta original realiza 4 `JOIN`, filtra por fecha y flags, agrega y ordena para obtener Top 5; con uso frecuente por panel en tiempo real este costo se vuelve notable (coincide con lo

observado en el plan).
- **Sincronización:** Se garantiza con **disparadores AFTER** sobre `detalle_pedido` (INSERT/UPDATE/DELETE) que recalculan la fila afectada por `(fecha, categoria_id)` usando `fn_recalcular_ventas_categoria_dia()` sobre la **fuente de verdad** (`detalle_pedido`, `producto`, `pedido`, `categoria`). También hay disparador sobre `pedido` para cambios de `fecha`/`eliminado`. Los valores se recalculan desde los datos originales (nunca se confía solo en incrementos ad-hoc).
- **Reversibilidad:** **Sí, sin pérdida de información.** La tabla es completamente redundante; se puede regenerar en cualquier momento con la consulta original, eliminarla y volver a usar el esquema normalizado sin perder datos transaccionales.

(c) Implementación
Ver `sql/tp_desnormalizacion_top_categorias.sql`:
- Creación de `ventas_categoria_dia` con índices
- Funciones `fn_recalcular_ventas_categoria_dia()` y `fn_trg_sync_ventas_categoria_dia()` / `fn_trg_sync_vcd_por_pedido()`
- Triggers AFTER sobre `detalle_pedido` y `pedido`
- Carga inicial desde fuente de verdad

(d) Consulta desnormalizada + comparación antes/después
Consulta desnormalizada:
```sql
SELECT categoria_nombre AS categoria,
      total_vendido
FROM ventas_categoria_dia
WHERE fecha = CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;
```

Tabla comparativa (plantilla):

Aspecto	Antes (normalizado, 4 JOINs + GROUP BY + ORDER BY)	Después (desnormalizado, tabla precalculada)
Tiempo de ejecución (EXPLAIN ANALYZE)	(registrar valor medido)	(registrar valor medido)
Nodo dominante del plan	(ej. Hash Join / Nested Loop + HashAggregate/GroupAggregate + Sort)	(ej. Index Scan/Bitmap Index Scan + Sort/Index scan por índice en (fecha,total_vendido DESC), sin joins masivos)

(Completar con las salidas reales de `EXPLAIN (ANALYZE, BUFFERS)` sobre la instancia usada.)

(e) Script de auditoría (resultado vacío)
Ver script en `sql/tp_desnormalizacion_top_categorias.sql` — compara `ventas_categoria_dia` con agregación desde fuente de verdad para `CURRENT_DATE`. Si no hay diferencias, retorna **0 filas**.

Entregables
- `sql/tp_fnbc_control_lote.sql` — Parte 1 completo
- `sql/tp_desnormalizacion_top_categorias.sql` — Parte 2 completo
- `informe/Informe_TP_U4_FNBC_Desnormalizacion_FoodStore.md` — este informe
- Repositorio: https://github.com/VIKROOM/DB-TPS-COM4.git

ANEXO A

```
-- ============================================================================
-- TP_U4_FNBC_Desnormalizacion_FoodStore - Parte 1
-- Archivo: tp_fnbc_control_lote.sql
-- ============================================================================
-- 1) ESQUEMA ORIGINAL (contexto) - control_lote_almacen
-- Tabla tal como se presenta en la consigna
-- ============================================================================

DROP TABLE IF EXISTS control_lote_almacen CASCADE;

CREATE TABLE control_lote_almacen (
    lote_id                 BIGINT NOT NULL, -- REFERENCES lote(id)
    deposito_id             BIGINT NOT NULL, -- REFERENCES deposito(id)
```

```
    responsable_control_id  BIGINT NOT NULL, -- REFERENCES usuario(id)
    PRIMARY KEY (lote_id, deposito_id)
);

-- Instancia de ejemplo provista
INSERT INTO control_lote_almacen VALUES
    (501, 30, 801),
    (502, 30, 801),
    (503, 31, 802);

-- ============================================================================
-- 2) ANÁLISIS (a)-(d) - Resuelto en informe, pero se documenta aquí
-- Dependencias funcionales (notación formal):
-- DF1: {lote_id, deposito_id} -> {responsable_control_id}
-- DF2: {responsable_control_id} -> {deposito_id}
--
-- Claves candidatas: {lote_id, deposito_id} (única razonable).
-- Atributos primos: lote_id, deposito_id. No primos: responsable_control_id.
-- Violación FNBC: determinante responsable_control_id no es superclave -> no está en FNBC.
-- Anomalías: inserción/borrado/actualización habilitadas por DF2.
-- ============================================================================

-- ============================================================================
-- 3) DESCOMPOSICIÓN SIN PÉRDIDA (R1,R2) + VISTA DE COMPATIBILIDAD
-- Se descompone en:
-- R1(responsable_control_id, deposito_id) con PK(responsable_control_id)
-- R2(lote_id, responsable_control_id) con PK(lote_id, responsable_control_id)
-- Vista v_control_lote_almacen recompone (lote_id, deposito_id, responsable_control_id)
-- ============================================================================

DROP VIEW IF EXISTS v_control_lote_almacen CASCADE;
DROP TABLE IF EXISTS control_lote CASCADE;
DROP TABLE IF EXISTS responsable_deposito CASCADE;

CREATE TABLE responsable_deposito (
    responsable_control_id  BIGINT NOT NULL,
    deposito_id             BIGINT NOT NULL,
    PRIMARY KEY (responsable_control_id)
);

CREATE TABLE control_lote (
    lote_id                 BIGINT NOT NULL,
    responsable_control_id BIGINT NOT NULL,
    PRIMARY KEY (lote_id, responsable_control_id),
    CONSTRAINT fk_control_lote_resp FOREIGN KEY (responsable_control_id)
        REFERENCES responsable_deposito(responsable_control_id)
);

-- Vista de compatibilidad que reconstruye la relación original mediante JOIN
CREATE OR REPLACE VIEW v_control_lote_almacen AS
SELECT cl.lote_id, rd.deposito_id, cl.responsable_control_id
FROM control_lote cl
JOIN responsable_deposito rd
  ON cl.responsable_control_id = rd.responsable_control_id;

-- ============================================================================
-- 4) MIGRACIÓN DE DATOS DESDE LA INSTANCIA DE EJEMPLO
-- ============================================================================

-- Migrar: primero responsables->depósitos (un responsable por depósito según regla)
INSERT INTO responsable_deposito (responsable_control_id, deposito_id)
SELECT DISTINCT responsable_control_id, deposito_id
FROM control_lote_almacen
ON CONFLICT (responsable_control_id) DO NOTHING;

-- Migrar: (lote, responsable)
INSERT INTO control_lote (lote_id, responsable_control_id)
SELECT lote_id, responsable_control_id
FROM control_lote_almacen
ON CONFLICT (lote_id, responsable_control_id) DO NOTHING;

-- ============================================================================
-- 5) VERIFICACIÓN DE EQUIVALENCIA MEDIANTE LA VISTA DE COMPATIBILIDAD
-- ============================================================================
```

```
-- Debe reconstruir exactamente la relación original
SELECT * FROM v_control_lote_almacen ORDER BY lote_id, deposito_id;

-- Comparación directa: debería dar 0 filas si son equivalentes
-- (proyección igual al original)
SELECT cla.lote_id, cla.deposito_id, cla.responsable_control_id
FROM control_lote_almacen cla
LEFT JOIN v_control_lote_almacen v
  ON cla.lote_id = v.lote_id AND cla.deposito_id = v.deposito_id AND cla.responsable_control_id =
v.responsable_control_id
WHERE v.lote_id IS NULL;

-- ============================================================================
-- JUSTIFICACIÓN (f): La unión es sin pérdida porque el atributo común
-- responsable_control_id es clave en responsable_deposito (PK) y aparece
-- como FK en control_lote; además rd(responsable_control_id)->(deposito_id)
-- garantiza unicidad, por lo que el JOIN por PK/FK reconstruye exactamente
-- (lote_id, deposito_id, responsable_control_id) sin generar duplicados.
-- ============================================================================
```

ANEXO B

```
-- ============================================================================
-- TP_U4_FNBC_Desnormalizacion_FoodStore - Parte 2
-- Archivo: tp_desnormalizacion_top_categorias.sql
-- ============================================================================

-- 5.1 Consulta original
-- SELECT c.nombre AS categoria,
--        SUM(dp.subtotal) AS total_vendido
-- FROM detalle_pedido dp
-- JOIN producto pr ON pr.id = dp.producto_id
-- JOIN categoria c ON c.id = pr.categoria_id
-- JOIN pedido ped ON ped.id = dp.pedido_id
-- WHERE ped.fecha = CURRENT_DATE
--   AND dp.eliminado = FALSE
--   AND ped.eliminado = FALSE
-- GROUP BY c.nombre
-- ORDER BY total_vendido DESC
-- LIMIT 5;

-- 5.2(a) Medición antes (ejecutar sobre instancia razonablemente poblada)
-- EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
-- ... misma consulta ...

-- ============================================================================
-- 5.2(b),(c) Desnormalización controlada elegida: COLUMNA PRECALCULADA CON DISPARADOR
-- Justificación (resumida): evidencia de EXPLAIN ANALYZE (muchos JOIN + agrupación)
-- motiva precálculo; disparadores AFTER INSERT/UPDATE/DELETE sobre detalle_pedido
-- y/o cambios en pedido mantienen la redundancia sincronizada con fuente de verdad
-- (detalle_pedido.subtotal + pedido.fecha/estado/eliminado). Es reversible sin
-- pérdida (se puede recalcular con la consulta original).
-- ============================================================================

-- Tabla desnormalizada: ventas por categoría por día
DROP TABLE IF EXISTS ventas_categoria_dia CASCADE;

CREATE TABLE ventas_categoria_dia (
    fecha                DATE        NOT NULL,
    categoria_id         BIGINT      NOT NULL,
    categoria_nombre     VARCHAR(80) NOT NULL,
    total_vendido        NUMERIC(12,2) NOT NULL DEFAULT 0,
    cantidad_items       BIGINT      NOT NULL DEFAULT 0,
    cantidad_pedidos     BIGINT      NOT NULL DEFAULT 0,
    actualizado_en       TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (fecha, categoria_id)
);

CREATE INDEX IF NOT EXISTS idx_vcd_fecha_total ON ventas_categoria_dia(fecha, total_vendido DESC);

-- Función para recalcular un día/categoría desde fuente de verdad
CREATE OR REPLACE FUNCTION fn_recalcular_ventas_categoria_dia(p_fecha DATE, p_categoria_id BIGINT)
```

```
RETURNS void AS $$
DECLARE
    v_nombre VARCHAR(80);
    v_total  NUMERIC(12,2);
    v_items  BIGINT;
    v_ped    BIGINT;
BEGIN
    SELECT c.nombre INTO v_nombre FROM categoria c WHERE c.id = p_categoria_id;

    SELECT COALESCE(SUM(dp.subtotal),0), COALESCE(SUM(dp.cantidad),0), COUNT(DISTINCT dp.pedido_id)
      INTO v_total, v_items, v_ped
    FROM detalle_pedido dp
    JOIN producto pr ON pr.id = dp.producto_id
    JOIN pedido ped ON ped.id = dp.pedido_id
    WHERE ped.fecha = p_fecha
      AND pr.categoria_id = p_categoria_id
      AND dp.eliminado = FALSE
      AND ped.eliminado = FALSE;

    INSERT INTO ventas_categoria_dia(fecha, categoria_id, categoria_nombre, total_vendido,
cantidad_items, cantidad_pedidos, actualizado_en)
    VALUES (p_fecha, p_categoria_id, COALESCE(v_nombre,''), v_total, v_items, v_ped, now())
    ON CONFLICT (fecha, categoria_id) DO UPDATE SET
        categoria_nombre = EXCLUDED.categoria_nombre,
        total_vendido    = EXCLUDED.total_vendido,
        cantidad_items   = EXCLUDED.cantidad_items,
        cantidad_pedidos = EXCLUDED.cantidad_pedidos,
        actualizado_en   = now();
END;
$$ LANGUAGE plpgsql;

-- Disparador: al cambiar detalle_pedido recalcula día/categoría afectada
CREATE OR REPLACE FUNCTION fn_trg_sync_ventas_categoria_dia()
RETURNS TRIGGER AS $$
DECLARE
    v_pedido_id BIGINT;
    v_fecha     DATE;
    v_cat       BIGINT;
BEGIN
    IF TG_OP = 'DELETE' THEN
        v_pedido_id := OLD.pedido_id;
    ELSE
        v_pedido_id := NEW.pedido_id;
    END IF;

    SELECT ped.fecha INTO v_fecha FROM pedido ped WHERE ped.id = v_pedido_id;
    IF v_fecha IS NULL THEN
        RETURN COALESCE(NEW, OLD);
    END IF;

    -- obtener categorías afectadas (OLD y/o NEW)
    IF TG_OP = 'INSERT' THEN
        SELECT pr.categoria_id INTO v_cat FROM producto pr WHERE pr.id = NEW.producto_id;
        IF v_cat IS NOT NULL THEN PERFORM fn_recalcular_ventas_categoria_dia(v_fecha, v_cat); END IF;
    ELSIF TG_OP = 'UPDATE' THEN
        -- si cambió producto/cantidad/subtotal/eliminado o pedido_id
        IF NEW.producto_id IS NOT NULL THEN
            SELECT pr.categoria_id INTO v_cat FROM producto pr WHERE pr.id = NEW.producto_id;
            IF v_cat IS NOT NULL THEN PERFORM fn_recalcular_ventas_categoria_dia(v_fecha, v_cat); END IF;
        END IF;
        IF OLD.producto_id IS DISTINCT FROM NEW.producto_id OR OLD.pedido_id IS DISTINCT FROM
NEW.pedido_id THEN
            SELECT ped.fecha INTO v_fecha FROM pedido ped WHERE ped.id = OLD.pedido_id;
            IF v_fecha IS NOT NULL THEN
                SELECT pr.categoria_id INTO v_cat FROM producto pr WHERE pr.id = OLD.producto_id;
                IF v_cat IS NOT NULL THEN PERFORM fn_recalcular_ventas_categoria_dia(v_fecha, v_cat); END
IF;
            END IF;
            SELECT ped.fecha INTO v_fecha FROM pedido ped WHERE ped.id = NEW.pedido_id;
            IF v_fecha IS NOT NULL THEN
                SELECT pr.categoria_id INTO v_cat FROM producto pr WHERE pr.id = NEW.producto_id;
                IF v_cat IS NOT NULL THEN PERFORM fn_recalcular_ventas_categoria_dia(v_fecha, v_cat); END
IF;
            END IF;
            RETURN NEW;
```

```
        END IF;
        -- recalcular categoría original si cambió eliminado o montos pero mismo producto/pedido
        SELECT pr.categoria_id INTO v_cat FROM producto pr WHERE pr.id = OLD.producto_id;
        IF v_cat IS NOT NULL THEN PERFORM fn_recalcular_ventas_categoria_dia(v_fecha, v_cat); END IF;
    ELSIF TG_OP = 'DELETE' THEN
        SELECT pr.categoria_id INTO v_cat FROM producto pr WHERE pr.id = OLD.producto_id;
        IF v_cat IS NOT NULL THEN PERFORM fn_recalcular_ventas_categoria_dia(v_fecha, v_cat); END IF;
    END IF;

    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_sync_vcd_ins ON detalle_pedido;
DROP TRIGGER IF EXISTS trg_sync_vcd_upd ON detalle_pedido;
DROP TRIGGER IF EXISTS trg_sync_vcd_del ON detalle_pedido;

CREATE TRIGGER trg_sync_vcd_ins
AFTER INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION fn_trg_sync_ventas_categoria_dia();

CREATE TRIGGER trg_sync_vcd_upd
AFTER UPDATE ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION fn_trg_sync_ventas_categoria_dia();

CREATE TRIGGER trg_sync_vcd_del
AFTER DELETE ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION fn_trg_sync_ventas_categoria_dia();

-- Sincronizar con cambios en pedido (fecha/eliminado) - recalcular todas las categorías del pedido
CREATE OR REPLACE FUNCTION fn_trg_sync_vcd_por_pedido()
RETURNS TRIGGER AS $$
DECLARE
    r RECORD;
BEGIN
    IF TG_OP = 'UPDATE' THEN
        IF OLD.fecha IS DISTINCT FROM NEW.fecha OR OLD.eliminado IS DISTINCT FROM NEW.eliminado THEN
            FOR r IN
                SELECT DISTINCT pr.categoria_id, COALESCE(NEW.fecha, OLD.fecha) AS f
                FROM detalle_pedido dp
                JOIN producto pr ON pr.id = dp.producto_id
                WHERE dp.pedido_id = NEW.id AND dp.eliminado = FALSE
            LOOP
                PERFORM fn_recalcular_ventas_categoria_dia(r.f, r.categoria_id);
                IF OLD.fecha IS DISTINCT FROM NEW.fecha AND OLD.fecha IS NOT NULL THEN
                    PERFORM fn_recalcular_ventas_categoria_dia(OLD.fecha, r.categoria_id);
                END IF;
            END LOOP;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_sync_vcd_pedido_upd ON pedido;
CREATE TRIGGER trg_sync_vcd_pedido_upd
AFTER UPDATE ON pedido
FOR EACH ROW EXECUTE FUNCTION fn_trg_sync_vcd_por_pedido();

-- Carga inicial
INSERT INTO ventas_categoria_dia(fecha, categoria_id, categoria_nombre, total_vendido, cantidad_items,
cantidad_pedidos, actualizado_en)
SELECT ped.fecha, pr.categoria_id, c.nombre, SUM(dp.subtotal), SUM(dp.cantidad), COUNT(DISTINCT
dp.pedido_id), now()
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id
JOIN pedido ped ON ped.id = dp.pedido_id
WHERE dp.eliminado = FALSE AND ped.eliminado = FALSE
GROUP BY ped.fecha, pr.categoria_id, c.nombre
ON CONFLICT (fecha, categoria_id) DO UPDATE SET
    categoria_nombre = EXCLUDED.categoria_nombre,
    total_vendido    = EXCLUDED.total_vendido,
    cantidad_items   = EXCLUDED.cantidad_items,
    cantidad_pedidos = EXCLUDED.cantidad_pedidos,
```

```
    actualizado_en   = now();

-- ============================================================================
-- 5.2(d) Consulta que lee directamente de la estructura desnormalizada
-- ============================================================================

-- Reemplaza la consulta original
SELECT categoria_nombre AS categoria,
       total_vendido
FROM ventas_categoria_dia
WHERE fecha = CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;

-- Medición después (ejecutar con EXPLAIN (ANALYZE, BUFFERS))
-- EXPLAIN (ANALYZE, BUFFERS)
-- SELECT categoria_nombre AS categoria, total_vendido
-- FROM ventas_categoria_dia
-- WHERE fecha = CURRENT_DATE
-- ORDER BY total_vendido DESC
-- LIMIT 5;

-- ============================================================================
-- 5.2(e) Script de auditoría - debe retornar vacío si sincronizado
-- ============================================================================

-- Detecta diferencias entre fuente de verdad y tabla desnormalizada para fecha actual
WITH verdad AS (
    SELECT ped.fecha, pr.categoria_id, c.nombre AS categoria_nombre,
           SUM(dp.subtotal) AS total_vendido,
           SUM(dp.cantidad) AS cantidad_items,
           COUNT(DISTINCT dp.pedido_id) AS cantidad_pedidos
    FROM detalle_pedido dp
    JOIN producto pr ON pr.id = dp.producto_id
    JOIN categoria c ON c.id = pr.categoria_id
    JOIN pedido ped ON ped.id = dp.pedido_id
    WHERE dp.eliminado = FALSE AND ped.eliminado = FALSE
      AND ped.fecha = CURRENT_DATE
    GROUP BY ped.fecha, pr.categoria_id, c.nombre
),
desnorm AS (
    SELECT fecha, categoria_id, categoria_nombre, total_vendido, cantidad_items, cantidad_pedidos
    FROM ventas_categoria_dia
    WHERE fecha = CURRENT_DATE
)
SELECT v.fecha, v.categoria_id,
       v.total_vendido AS verdad_total, d.total_vendido AS desnorm_total,
       v.cantidad_items AS verdad_items, d.cantidad_items AS desnorm_items,
       v.cantidad_pedidos AS verdad_ped, d.cantidad_pedidos AS desnorm_ped
FROM verdad v
FULL OUTER JOIN desnorm d
  ON v.fecha = d.fecha AND v.categoria_id = d.categoria_id
WHERE COALESCE(v.total_vendido,0) <> COALESCE(d.total_vendido,0)
   OR COALESCE(v.cantidad_items,0) <> COALESCE(d.cantidad_items,0)
   OR COALESCE(v.cantidad_pedidos,0) <> COALESCE(d.cantidad_pedidos,0)
   OR v.categoria_nombre IS DISTINCT FROM d.categoria_nombre;

-- ============================================================================
-- NOTAS:
-- - Patrón elegido: columna precalculada con disparador (trigger-based materialization)
-- - Garantiza sincronización con fuente de verdad vía triggers AFTER sobre detalle_pedido/pedido
-- - Reversible sin pérdida: se puede recalcular con consulta original en cualquier momento
-- ============================================================================
```